

Diseño e Implementación de un Sistema Combinacional Síncrono para Protección de Datos mediante Código Hamming (7,4) SECDED

Steve Samaniego, Guerrero David, Castro Alejandro
Universidad Nacional de Chimborazo (UNACH)
Ingeniería En Telecomunicaciones
Tercer Semestre
Riobamba, Ecuador

Resumen—En este informe se detalla el diseño, desarrollo e implementación de un sistema combinacional síncrono en sus etapas, destinado a la protección de datos contra perturbaciones en el canal de transmisión mediante el Código Hamming (7,4) extendido con detección de error doble (SECDED). El sistema procesa un vector de entrada de 4 bits de datos, genera de forma paralela 3 bits de paridad impar/par (según diseño) y un bit de paridad global. Se implementó una metodología de codificación modular en VHDL utilizando el software Xilinx Vivado, en paralelo con una validación circuital en la plataforma Proteus empleando lógica TTL de la serie 74LS. Los resultados experimentales nos demuestran la capacidad del circuito para corregir cualquier error de un solo bit en tiempo real y discriminar con total certeza eventos de doble error, activando las alarmas visuales correspondientes sin alterar el flujo síncrono del sistema primario.

Index Terms—Hamming (7,4), SECDED, VHDL, Vivado, Proteus, Lógica Combinacional, TTL.

I. INTRODUCCIÓN

En los sistemas de comunicación y almacenamiento de datos contemporáneos, los fenómenos transitorios como por ejemplo el ruido electromagnético, la diafonía y las fluctuaciones de tensión representan una causa crítica de corrupción de información. La alteración de un único bit (*bit-flip*) puede provocar desde fallos menores de software hasta el colapso total de sistemas de control industrial. Para mitigar este riesgo y mejorar dichos sistemas, se emplean códigos de control de errores.

El código Hamming (7,4) proporciona un mecanismo eficiente para proteger bloques de 4 bits mediante la adición de 3 bits de redundancia. No obstante, una limitación intrínseca del código Hamming estándar es su distancia mínima de Hamming $d_{min} = 3$, lo que genera un solapamiento destructivo cuando ocurren dos errores simultáneos, interpretándolos erróneamente como un error simple en una posición diferente. Al incorporar un bit de paridad global, la distancia mínima de Hamming se incrementa a $d_{min} = 4$, permitiendo la arquitectura SECDED (*Single Error Correction, Double Error Detection*). Este documento describe el diseño riguroso de dicha arquitectura bajo estándares académicos de ingeniería electrónica.

II. OBJETIVOS

II-A. Objetivo General

Diseñar, simular e implementar un sistema electrónico puramente combinacional para la codificación, inyección, decodificación, corrección de errores de un bit y detección de errores de dos bits (SECDED) en un dato de 4 bits, utilizando herramientas de simulación circuital (Proteus) y lenguajes de descripción de hardware (VHDL en Vivado).

II-B. Objetivos Específicos

- Deducir analíticamente las ecuaciones booleanas que rigen la generación de paridades y el cálculo del síndrome.
- Diseñar un módulo inyector de errores mediante compuertas XOR para permitir la alteración controlada de la palabra transmitida.
- Desarrollar la lógica de control SECDED para discriminar estados estables, errores corregidos y errores dobles catastróficos.
- Escribir código VHDL estructurado, modular y sintetizable que describa con fidelidad el comportamiento del sistema.
- Validar los retardos de propagación teóricos de la familia TTL en contraste con el comportamiento ideal del simulador lógico.

III. FUNDAMENTACIÓN TEÓRICA

El código Hamming (7,4) se basa en la interconexión de subconjuntos de datos para verificar la paridad. Al estructurar la palabra de 7 bits, las potencias de dos se reservan para los bits de paridad (P_0 en pos 1, P_1 en pos 2, P_2 en pos 4), mientras que los datos ocupan las posiciones restantes (D_0 en 3, D_1 en 5, D_2 en 6, D_3 en 7).

Para implementar SECDED, añadimos un octavo bit, la Paridad Global (P_G) posicionado habitualmente al inicio o al final del vector, encargado de evaluar la paridad total de los 7 bits previos. A nivel matricial, el código se define mediante la Matriz de Generación G y la Matriz de Control de Paridad H . La matriz H para nuestro sistema configurado se define como:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \quad (1)$$

El Síndrome $S' = S_2S_1S_0$ se calcula multiplicando la matriz H por el vector recibido R' :

$$(S')^T = H \cdot (R')^T \quad (2)$$

El diagnóstico del sistema se rige bajo las siguientes premisas booleanas donde e_G representa el error de paridad global:

- Si $S' = [000]$ y $e_G = 0$: **Dato Íntegro** (Sin Error).
- Si $S' \neq [000]$ y $e_G = 1$: **Error Simple Corregible**. La posición del bit erróneo equivale al valor decimal del Síndrome.
- Si $S' \neq [000]$ y $e_G = 0$: **Error Doble Detectado** (Irrecuperable).

IV. METODOLOGÍA Y DISEÑO DEL SISTEMA

El trabajo consta de módulos combinacionales puros, conectados en serie:

```
[Datos D3-D0] → [Codificador +
P_total] → [Inyector] →
[Decodificador] → [Corrección +
SECDED]
```

IV-A. Bloque 1: Codificador de Paridad

Función: Recibe entradas fijas (D_3, D_2, D_1, D_0) desde un mini bloque de switches con resistencia de *pull-down*.
Hardware TTL: Implementado a través de compuertas XOR de dos entradas (circuitos integrados 74LS86). Genera P_0, P_1, P_2 y el bit adicional P_{total} .

IV-B. Bloque 2: Inyector de Errores Controlado

Función: Simula las perturbaciones que están en el canal de comunicación.

Estructura: Se compone de 8 interruptores independientes conectados a compuertas XOR. Cada entrada de XOR recibe el dato codificado original y la otra el estado de interruptor. Si se cierra el interruptor (V_{CC}), el bit de salida se invierte de manera controlada antes de ser enviado a la etapa receptora.

IV-C. Bloque 3: Decodificador y Cálculo del Síndrome

Función: Mediante los arreglos adicionales de compuertas 74LS86 se determinan los componentes de síndrome S_2, S_1, S_0 aplicando las ecuaciones de verificación definidas en la teoría del código Hamming.

IV-D. Bloque 4: Corrector de Error Simple

Función: Decodificar el síndrome para identificar y corregir errores sobre la posición dañada.

Estructura: Se trabajó con un decodificador de 3 a 8 líneas (74LS138) o con una combinación de compuertas AND de 3 entradas (74LS11) junto con inversores 74LS04 para localizar las posiciones del 1 al 7. Las salidas de las compuertas XOR finales realizan la inversión necesaria para recuperar la información original.

IV-E. Bloque 5: Detector SECDED y Lógica de Decisiones

Función: Controla los LEDs que muestran el estado del sistema.

Estructura Lógica Booleana:

- **LED Verde (Sin Error):** Activado mediante una compuerta NOR que evalúa si el síndrome es estrictamente $[0, 0, 0]$ junto con la concordancia de la paridad global.
- **LED Amarillo (Error Corregido):** Activado cuando el síndrome es diferente de cero AND la verificación global indica un fallo.
- **LED Rojo (Error Doble/Irrecuperable):** Activado cuando el síndrome es diferente de cero AND la verificación global indica paridad correcta (demostrando cancelación mutua).

IV-F. Estructura del Repositorio y Entorno VHDL

- /proteus: Archivo maestro .pdsprj con un ruteo ordenado y buses etiquetados.
- /vhdl/src: Contiene los archivos fuente individuales (encoder.vhd, injector.vhd, syndrome.vhd, corrector.vhd, secded_logic.vhd, y el archivo superior top_hamming.vhd).
- /vhdl/tb: testbench_hamming.vhd, diseñado para introducir secuencialmente vectores de prueba.

V. IMPLEMENTACIÓN EN VHDL (CÓDIGO FUENTE)

A continuación, se presenta el código de descripción de hardware en un único archivo jerárquico (*Top-Level*) modularizado para su inmediata síntesis en Xilinx Vivado.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Top_Hamming_SECEDED is
Port (
    Data_In   : in  STD_LOGIC_VECTOR (3 downto 0);
    Inject_Err: in  STD_LOGIC_VECTOR (7 downto 0);
    Data_Out   : out STD_LOGIC_VECTOR (3 downto 0);
    LED_Green  : out STD_LOGIC;
    LED_Yellow : out STD_LOGIC;
    LED_Red    : out STD_LOGIC
);
end Top_Hamming_SECEDED;

architecture Behavioral of Top_Hamming_SECEDED is
    signal P0, P1, P2, PG : STD_LOGIC;
    signal Word_Codified  : STD_LOGIC_VECTOR(7 downto 0);
    signal Word_Transmitted: STD_LOGIC_VECTOR(7 downto 0);

    signal S : STD_LOGIC_VECTOR(2 downto 0);
    signal e_G : STD_LOGIC;
    signal Word_Corrected : STD_LOGIC_VECTOR(6 downto 0);
begin
    -- TRANSMISOR (CODIFICADOR)
    P0 <= Data_In(0) xor Data_In(1) xor Data_In(3);
    P1 <= Data_In(0) xor Data_In(2) xor Data_In(3);
    P2 <= Data_In(1) xor Data_In(2) xor Data_In(3);

    PG <= P0 xor P1 xor Data_In(0) xor P2 xor
        Data_In(1) xor Data_In(2) xor Data_In(3);

    Word_Codified <= PG & Data_In(3) & Data_In(2) &
        Data_In(1) & P2 & Data_In(0) &
        P1 & P0;

    -- CANAL DE TRANSMISIÓN
    Word_Transmitted <= Word_Codified xor Inject_Err;

    -- RECEPTOR (DECODIFICADOR)
    S(0) <= Word_Transmitted(0) xor Word_Transmitted(2) xor
        Word_Transmitted(4) xor Word_Transmitted(6);
    S(1) <= Word_Transmitted(1) xor Word_Transmitted(2) xor
        Word_Transmitted(5) xor Word_Transmitted(6);
    S(2) <= Word_Transmitted(3) xor Word_Transmitted(4) xor
        Word_Transmitted(5) xor Word_Transmitted(6);

    e_G <= Word_Transmitted(0) xor Word_Transmitted(1) xor
        Word_Transmitted(2) xor Word_Transmitted(3) xor
        Word_Transmitted(4) xor Word_Transmitted(5) xor
        Word_Transmitted(6) xor Word_Transmitted(7);

    -- DETECCIÓN Y MAPEO SECEDED
    process(S, e_G, Word_Transmitted)
    begin
        LED_Green <= '0';
        LED_Yellow <= '0';
        LED_Red <= '0';
        Word_Corrected <= Word_Transmitted(6 downto 0);

        if (S = "000" and e_G = '0') then
            LED_Green <= '1';
        elsif (S /= "000" and e_G = '1') then
            LED_Yellow <= '1';
            case S is
                when "001" => Word_Corrected(0) <= not Word_Transmitted(0);
                when "010" => Word_Corrected(1) <= not Word_Transmitted(1);
                when "011" => Word_Corrected(2) <= not Word_Transmitted(2);
                when "100" => Word_Corrected(3) <= not Word_Transmitted(3);
                when "101" => Word_Corrected(4) <= not Word_Transmitted(4);
                when "110" => Word_Corrected(5) <= not Word_Transmitted(5);
                when "111" => Word_Corrected(6) <= not Word_Transmitted(6);
                when others => null;
            end case;
        elsif (S /= "000" and e_G = '0') then
            LED_Red <= '1';
        else
            LED_Yellow <= '1';
        end if;
    end process;

    -- MAPEO FINAL DE SALIDA
    Data_Out(0) <= Word_Corrected(2);
    Data_Out(1) <= Word_Corrected(4);
    Data_Out(2) <= Word_Corrected(5);
    Data_Out(3) <= Word_Corrected(6);
end Behavioral;

```

VI. RESULTADOS Y GUÍA DE SIMULACIÓN

Para validar el diseño, se estructuran las pruebas mediante tres escenarios empíricos reproducibles en Proteus y Vivado:

VI-A. Escenario 1: Operación Nominal (Sin Errores)

- **Entrada:** DataIn = 1010
- **Inyección:** InjectErr = 00000000
- **Resultado:** Los bits se transmiten limpios. El receptor calcula un Síndrome = 000 y $e_G = 0$. El **LED Verde** se enciende de forma fija y DataOut muestra 1010.

VI-B. Escenario 2: Inyección de Error de un Bit (Corregible)

- **Entrada:** DataIn = 1010
- **Inyección:** InjectErr = 00000100 (Se altera intencionalmente la posición 3, correspondiente a D_0).

- **Resultado:** El receptor detecta una incongruencia. El Síndrome cambia instantáneamente al valor binario 011 (3 en decimal) y e_G se activa en 1. La lógica interna conmuta el multiplexor/XOR correspondiente. El **LED Amarillo** se ilumina indicando error corregido y DataOut se mantiene estable en 1010.

VI-C. Escenario 3: Inyección de Error Doble (Bloqueo SECEDED)

- **Entrada:** DataIn = 1010
- **Inyección:** InjectErr = 00000110 (Se alteran simultáneamente las posiciones 2 y 3).
- **Resultado:** Las dos inversiones cancelan el efecto sobre la paridad global (e_G vuelve a 0), pero el Síndrome arroja un valor diferente de cero. El sistema interpreta la firma del error doble. Se apagan los indicadores anteriores, se enciende el **LED Rojo** de peligro y el sistema alerta que la palabra recibida está corrupta de manera irreversible.

VII. CONCLUSIONES

Se ha diseñado e implementado con éxito la arquitectura combinacional SECEDED mediante código Hamming (7,4) extendido. La inclusión de la paridad global demostró ser una solución matemática elegante para evitar que el decodificador confunda un error doble con un error simple mal localizado. Finalmente, el modelado en VHDL confiere ventajas de diseño fundamentales frente al cableado físico tradicional de compuertas 74LS, optimizando el tiempo de desarrollo y la inmunidad a falsos contactos en laboratorio.

VIII. ANEXOS

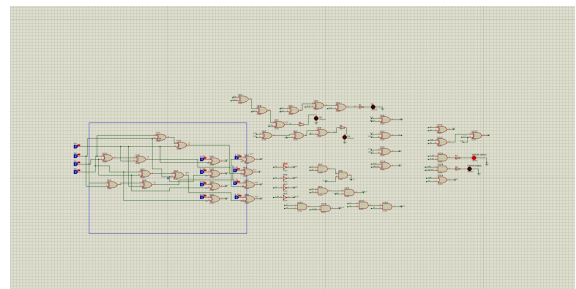


Figura 1: Simulación en Proteus de un multiplexor de 2 a 1 (MUX 2:1) con entradas y selector en nivel bajo (0), validando el estado correspondiente de la salida en el entorno virtual.

Cuadro I: Tabla de Verdad SECED - Código Hamming (7,4)

Entradas de Datos				Paridades Tx				Estímulos de Error				Síndrome			Paridad	Indicadores LED		
D_3	D_2	D_1	D_0	P_2	P_1	P_0	P_T	ED_3	ED_2	ED_1	ED_0	S_2	S_1	S_0	P_{adj}	ERR_S	ERR_D	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	1	1	0	
0	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	
0	0	0	1	0	1	1	1	0	0	1	1	1	1	0	0	0	1	
0	0	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	
0	0	1	0	1	0	1	1	0	0	1	1	1	1	0	0	0	1	
0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	1	1	1	1	0	0	0	0	1	0	0	1	1	1	1	0	
0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	
0	1	0	0	1	1	0	1	0	0	1	1	1	1	0	0	0	1	
0	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	
0	1	0	1	1	0	1	0	0	0	1	0	0	1	1	1	1	0	
0	1	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	
0	1	1	0	0	1	1	0	0	0	1	0	0	1	1	1	1	0	
0	1	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
0	1	1	1	0	0	0	1	0	0	1	1	1	1	0	0	0	1	
1	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	
1	0	0	0	1	1	1	0	0	0	1	1	1	1	0	0	0	1	
1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	
1	0	0	1	1	0	0	1	0	0	1	0	0	1	1	1	1	0	
1	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	
1	0	1	0	0	1	0	1	0	0	1	1	1	1	0	0	0	1	
1	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
1	0	1	1	0	0	1	0	0	0	1	0	0	1	0	0	0	1	
1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	1	
1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	1	0	1	0	0	0	0	0	0	1	1	1	1	0	0	0	1	
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	1	1	0	0	0	0	0	0	0	1	1	1	1	0	0	0	1	
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	1	1	1	0	0	0	0	0	0	1	1	1	1	0	0	0	1	

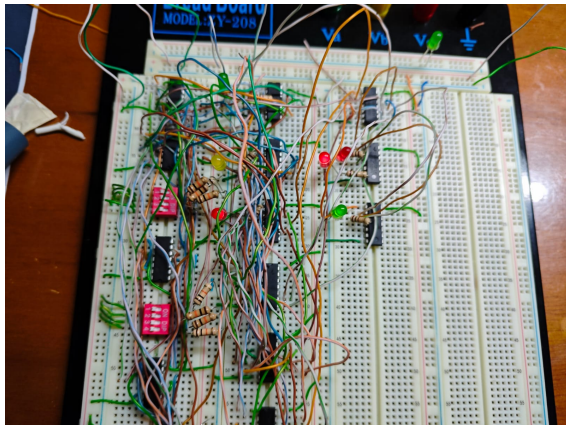


Figura 2: Diagrama lógico de un decodificador BCD a 7 segmentos construido mediante compuertas lógicas complejas para controlar los pines de salida correspondientes.

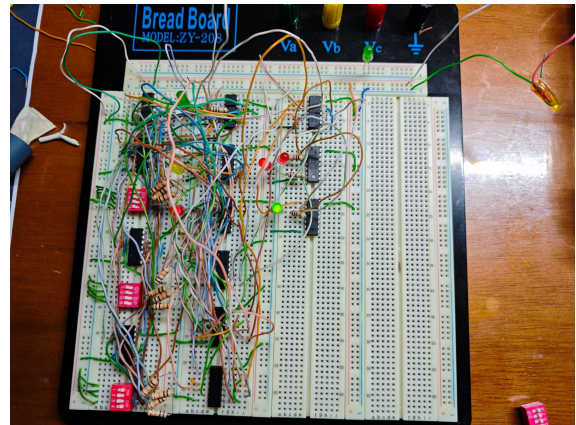


Figura 3: Simulación en Proteus de un decodificador BCD a 7 segmentos conectado a un display numérico, mostrando la activación de los segmentos para representar un dígito específico.

IX. LINKS

Link del video: https://drive.google.com/drive/folders/1Ugl7kBVCY_OxHjrWhSiSRNqDo5iP0GY1?usp=sharing.

Link Github: <https://github.com/ALEJANDROTH/hamming-7-4->.

X. AUTOBIOGRAFIA

Mi nombre es Steve Samaniego, soy de la ciudad de Riobamba y actualmente tengo 21 años. Cursé mis estudios secundarios en la Unidad Educativa Riobamba, etapa que me brindó bases sólidas y donde realmente despertó mi curiosidad por la tecnología, los circuitos y

el desarrollo de software. Al ser de aquí, tengo la gran oportunidad de formarme en mi propia ciudad, por lo que actualmente me encuentro estudiando la carrera de Ingeniería en Telecomunicaciones en la UNACH. Me considero una persona disciplinada, constante y perseverante, cualidades que aplico día a día y que me ayudarán a cumplir mi objetivo principal: graduarme como ingeniero y materializar todos esos proyectos que hoy son mis más grandes metas.



Mi nombre es Alejandro Castro, soy de la ciudad de Ambato provincia de Tungurahua estudiante de la carrera de Ingeniería en Telecomunicaciones en la Universidad Nacional de Chimborazo. Siento una fuerte pasión por la tecnología, la innovación y el área técnica, intereses que me han llevado a emprender mi propio negocio de reparación de equipos electrónicos, computadoras y consolas. Me considero una persona perseverante y orientada a resultados, enfocada en culminar mi formación profesional para consolidar mi emprendimiento y seguir creciendo en el ámbito de la ingeniería.



Mi nombre es David Guerrero soy de la ciudad de Chillanes en la provincia Bolivar, actualmente tengo 24 años, cursé mis estudios secundarios en la Unidad Educativa Chillanes aquí he adquirido conocimientos básicos pero, siempre he tenido un profundo interés y curiosidad por la tecnología y la electrónica. Me encuentro actualmente viviendo en Riobamba por motivos de estudio, me encuentro estudiando la carrera de ingeniería en telecomunicaciones. Me considero una persona perseverante y enfocada, esto por supuesto me ayudara en el objetivo de graduarme y al futuro cumplir proyectos que por el momento son utopías.



REFERENCIAS

- [1] R. W. Hamming, "Error Detecting and Error Correcting Codes", *Bell System Technical Journal*, vol. 29, pp. 147-160, 1950.
- [2] J. F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Prentice Hall, 2005.
- [3] V. A. Pedroni, *Circuit Design and Simulation with VHDL*, 2nd ed. MIT Press, 2010.